Allegiant Model Analysis

Introduction and Company Overview

Company Background

Allegiant Travel Company (NASDAQ: ALGT) is a U.S. ultra-low-cost carrier (ULCC) headquartered in **Las Vegas, Nevada**. Founded in **1997**, it connects **underserved regional airports** with leisure destinations (e.g., Las Vegas, Orlando, Phoenix) using **point-to-point**, low-frequency schedules that avoid congested hubs. Beyond airfare, Allegiant sells **bundled vacation products** (hotels, car rentals, entertainment) through *Allegiant Vacations*, generating substantial ancillary revenue while keeping base fares low.

As of 2025, Allegiant operates **120+ Airbus A320-family aircraft**, employs ~**5,000** people, and carries **~17 million** passengers annually. Its mission is to provide **affordable, convenient leisure travel** with strict cost discipline.

Motivation for Selected Variables

We model two metrics capturing demand and revenue:

1) Average Seats Filled (Load Factor) — Utilization/Efficiency

Load Factor = RPM / ASM × 100, where **RPM (Revenue Passenger Miles) = passengers × miles flown**. With passenger and ASM forecasts, we infer future utilization and assess whether capacity is tight (higher LF) or loose (excess seats), a driver of unit costs and profitability. In theory we do not want to see a significant model within this area. This is because Allegiant will try to model their available seat miles to the actual revenue passenger miles that they expect to be booked. If we do see a significant model within this statistic, it means that Allegiant is failing to consider a significant factor in that analysis which is then bleeding over into the average seats filled.

2) Average Revenue per Passenger — Monetization

Total Revenue/Passengers. This tracks how much value can be extracted per passenger between fare and fees. It is important to understand if there are any significant trends or seasonality because it alongside passenger count determines top of the line revenues. If there are any significant trends, it could indicate that some revenue could be gained without necessarily having to increase passenger counts.

Data Acquisition and Preparation:

Data was acquired from the quarterly reports issued by Allegiant. We used service revenue divided by passengers to get average revenue per passenger. We took passenger seat miles divided by available seat miles to derive the fill rate.

```r
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✓ dplyr     1.1.4     ✓ readr     2.1.5
✓ forcats   1.0.0     ✓ stringr   1.5.2
✓ ggplot2   4.0.0     ✓ tibble    3.3.0
✓ lubridate 1.9.4     ✓ tidyr     1.3.1
✓ purrr     1.1.0
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag()    masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
errors
```

```r
library(ggthemes)
library(fpp3)
```

```
Registered S3 method overwritten by 'tsibble':
  method               from
  as_tibble.grouped_df dplyr
── Attaching packages ──────────────────────────────────────────── fpp3 1.0.2 ──
✓ tsibble     1.1.6     ✓ feasts      0.4.2
✓ tsibbledata 0.4.1     ✓ fable       0.4.1
── Conflicts ───────────────────────────────────────────────── fpp3_conflicts ──
✗ lubridate::date()    masks base::date()
✗ dplyr::filter()      masks stats::filter()
✗ tsibble::intersect() masks base::intersect()
✗ tsibble::interval()  masks lubridate::interval()
✗ dplyr::lag()         masks stats::lag()
✗ tsibble::setdiff()   masks base::setdiff()
✗ tsibble::union()     masks base::union()
```

```r
library(tidyverse)
library(gt)
data<- read_csv('Allegiant.csv')
```

```
Rows: 41 Columns: 6
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): Quarter
dbl (5): Passenger_Seat_Miles, Available_Seat_Miles, Fuel_Per_ASM, Passenger...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
data  %>%
     mutate(Quarter = yearquarter(Quarter))%>%
     as_tsibble(index = Quarter)-> data_ts
```

```r
data_ts <- data_ts %>%
  mutate(average_charge = `Service Revenue`/Passengers)%>%
  mutate(fill = Passenger_Seat_Miles/Available_Seat_Miles)

data_ts %>%
  select(Quarter, average_charge, fill)%>%
     gt() %>%
     tab_header(
          title = "Allegiant Quarterly Data",
          subtitle = "Average Ticket Price and Fulfilment"
     )
```

Allegiant Quarterly Data

Average Ticket Price and Fulfilment

Quarter	average_charge	fill
2015 Q1	135.44600	0.8675539
2015 Q2	121.70713	0.8574993
2015 Q3	114.65376	0.8606533
2015 Q4	121.56511	0.8150287
2016 Q1	110.41842	0.8385761
2016 Q2	110.41842	0.8385761
2016 Q3	103.65985	0.8474731
2016 Q4	99.20943	0.7961906
2017 Q1	119.27540	0.8020814
2017 Q2	110.70739	0.8255121
2017 Q3	103.05939	0.8300493
2017 Q4	110.83642	0.8063935
2018 Q1	120.12621	0.8300262
2018 Q2	109.49234	0.8353782
2018 Q3	101.34569	0.8335849

Allegiant Quarterly Data

Average Ticket Price and Fulfilment

Quarter	average_charge	fill
2018 Q4	116.74838	0.7437457
2019 Q1	121.72266	0.8256769
2019 Q2	109.07185	0.8217483
2019 Q3	102.78089	0.8154578
2019 Q4	116.28024	0.7596252
2020 Q1	109.95702	0.5397167
2020 Q2	91.51327	0.4987196
2020 Q3	90.22533	0.4868999
2020 Q4	115.68603	0.7829258
2021 Q1	109.95702	0.5397167
2021 Q2	119.95701	0.6939136
2021 Q3	109.43305	0.7436094
2021 Q4	121.73515	0.7445809
2022 Q1	124.24436	0.7701156
2022 Q2	125.01142	0.8552614
2022 Q3	118.47364	0.8583884
2022 Q4	142.51756	0.8210083
2023 Q1	146.86848	0.8391790
2023 Q2	135.14499	0.8466131
2023 Q3	120.28128	0.8444794
2023 Q4	134.14200	0.8012163
2024 Q1	141.28034	0.8138796
2024 Q2	128.62799	0.8194927

Allegiant Quarterly Data Average Ticket Price and Fulfilment		
Quarter	average_charge	fill
2024 Q3	114.88731	0.8223287
2024 Q4	138.41294	0.7683909
2025 Q1	138.55484	0.7835022

Looking at the data, both statistics appear to randomly fluctuate by quarter, but there is a big dip in both statistics during COVID. This will be easier to see in later visualizations, and we need to keep it in mind when building our models.

Data Preparation

```
library(tidyverse)
library(ggthemes)
library(fpp3)

library(tidyverse)
data<- read_csv('Allegiant.csv')
```

```
Rows: 41 Columns: 6
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): Quarter
dbl (5): Passenger_Seat_Miles, Available_Seat_Miles, Fuel_Per_ASM, Passenger...

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

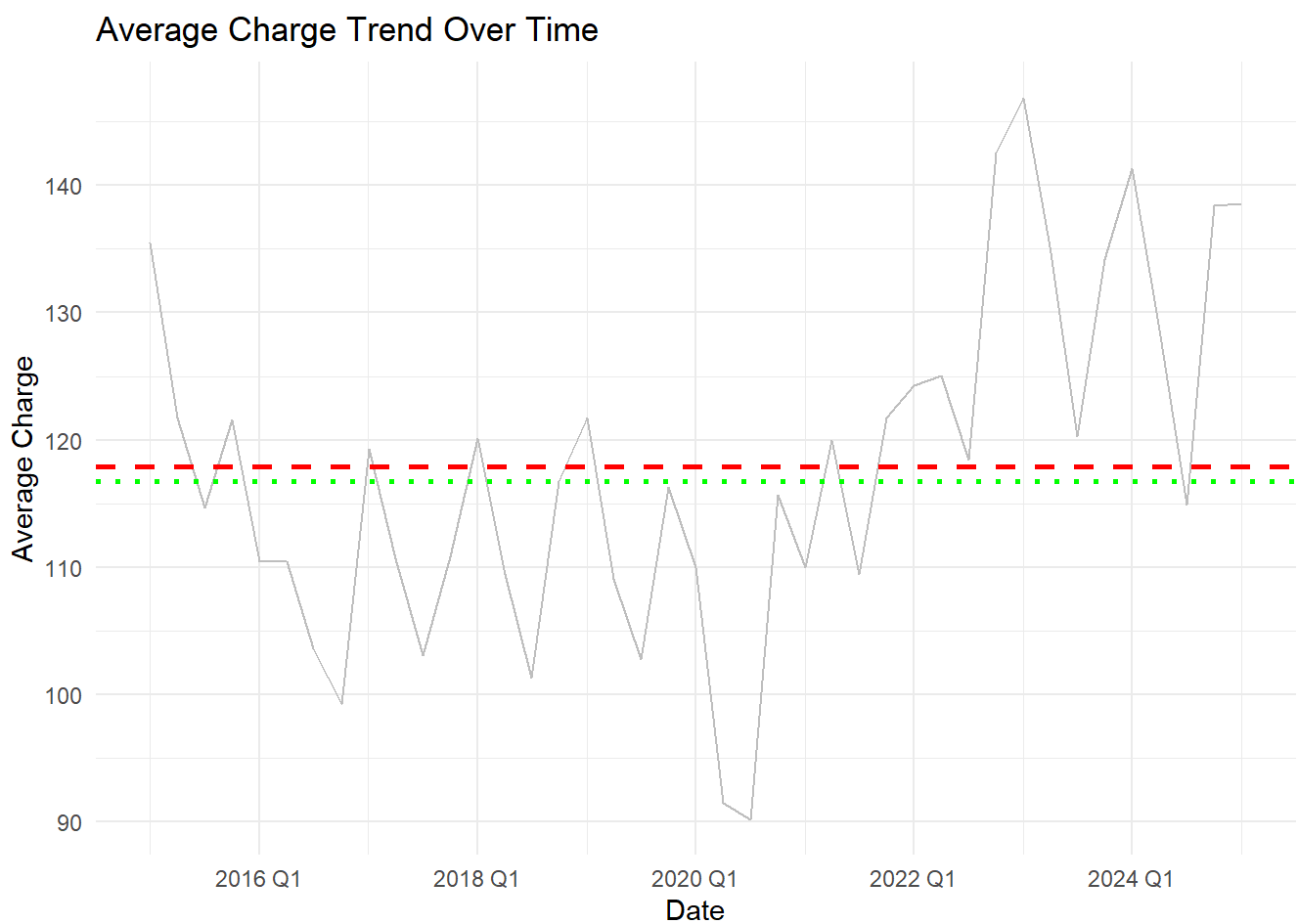
```
data %>% mutate(Quarter = yearquarter(Quarter))%>%
  as_tsibble(index = Quarter)-> data_ts

data_ts <- data_ts %>%
  mutate(average_charge = `Service Revenue`/Passengers)%>%
  mutate(fill = Passenger_Seat_Miles/Available_Seat_Miles)

train_ts <- filter_index(.data = data_ts, .~'2022 Q2')
test_ts <- filter_index(.data = data_ts, '2022 Q3' ~.)
```

Descriptive Stats

Looking at the Average revenue attributed to the amount charged to the passenger, the chart indicates the trend is very seasonal. With consistent peaks in Q1 of every year.

```
ggplot(data_ts, aes(x = Quarter, y = average_charge)) +
  geom_line(color = "gray") +
  geom_hline(aes(yintercept = mean(average_charge, na.rm = TRUE)),
             color = "red", linetype = "dashed", size = 1) +
  geom_hline(aes(yintercept = median(average_charge, na.rm = TRUE)),
             color = "green", linetype = "dotted", size = 1) +
  labs(title = "Average Charge Trend Over Time",
       x = "Date", y = "Average Charge") +
  theme_minimal()
```

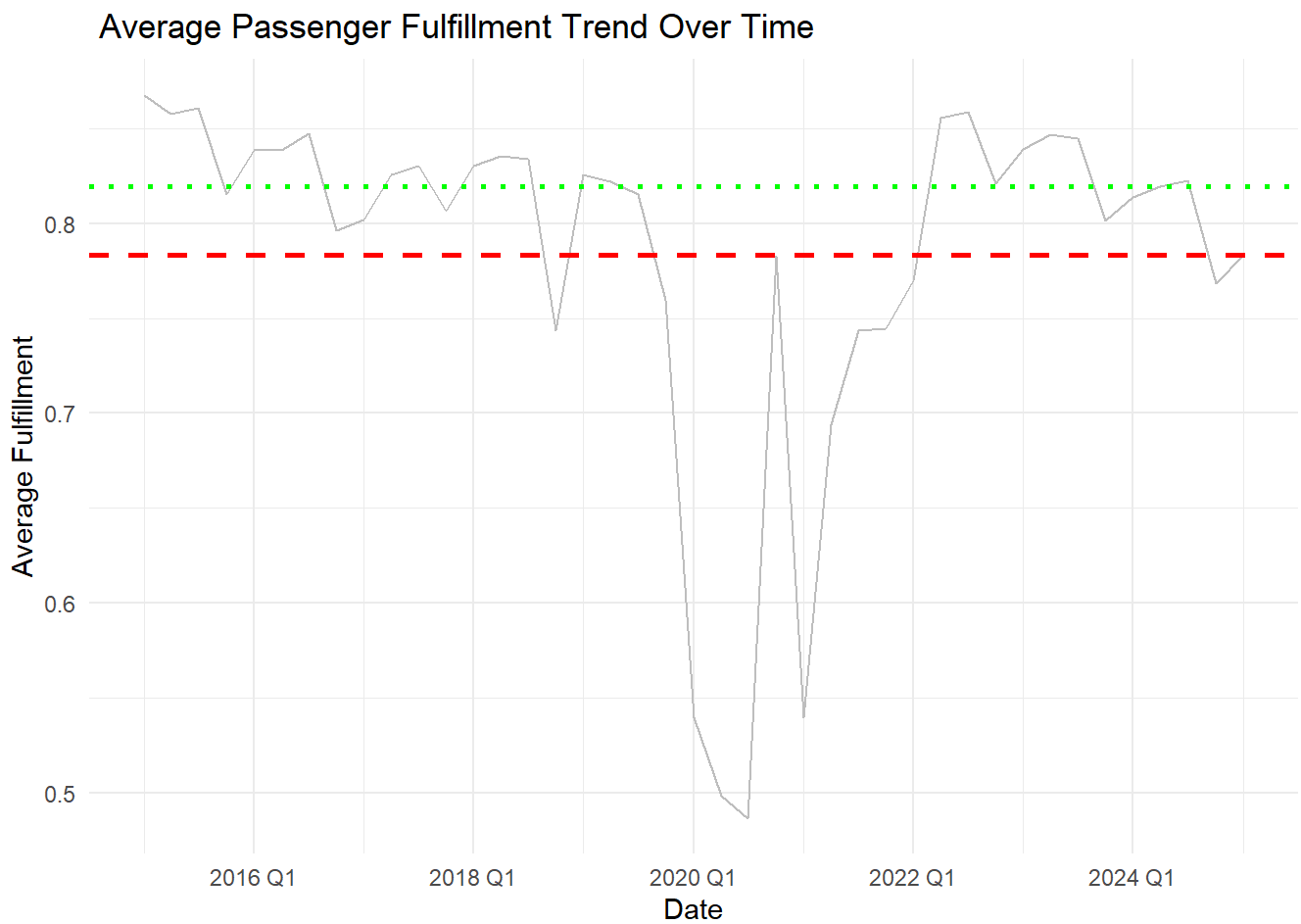
```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
ℹ Please use `linewidth` instead.
```



The fulfillment plots indicate how there is no significant trend pattern between the seasons. There is however a significant drop in fill rates during the Covid years.

```
ggplot(data_ts, aes(x = Quarter, y = fill)) +
  geom_line(color = "gray") +
  geom_hline(aes(yintercept = mean(fill, na.rm = TRUE)),
             color = "red", linetype = "dashed", size = 1) +
```

```
  geom_hline(aes(yintercept = median(fill, na.rm = TRUE)),
             color = "green", linetype = "dotted", size = 1) +
  labs(title = " Average Passenger Fulfillment Trend Over Time",
       x = "Date", y = "Average Fulfillment") +
  theme_minimal()
```



Both statistics are clearly heavily impacted by COVID during 2021, but otherwise their values do not deviate from the mean by much quarter to quarter. We may need to consider removing the COVID data as a test during our model building process.

Model Building

While the time series of our two variables looked mostly seasonal, we tested most variations of models. For average revenue per passenger, we first wanted to see if there was any significant relationship between average revenue per passenger and average fuel cost per seat mile. There may be a relationship because we may be able to pass some fuel cost onto the customer.

```
charge_fit <- train_ts %>%
  model(tslm = TSLM(average_charge ~ trend() + season()),
      tslmvar = TSLM(average_charge ~ trend() + season() + Fuel_Per_ASM))

charge_fit %>%
```

```
  glance()%>%
  select(.model, AIC, AICc, BIC)
```

```
# A tibble: 2 × 4
  .model    AIC  AICc   BIC
  <chr>   <dbl> <dbl> <dbl>
1 tslm     138.  142.  146.
2 tslmvar  140.  145.  149.
```

```
charge_fit %>% forecast(test_ts)%>% accuracy(data_ts)
```

```
# A tibble: 2 × 10
  .model  .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE   ACF1
  <chr>   <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 tslm    Test   22.2  22.7  22.2  16.6  16.6  2.62  1.91 0.0445
2 tslmvar Test   22.0  22.5  22.0  16.4  16.4  2.60  1.89 0.0658
```

```
charge_fit %>% forecast(test_ts) %>%
  accuracy(data_ts, list(winkler = winkler_score))
```

```
# A tibble: 2 × 3
  .model  .type winkler
  <chr>   <chr>   <dbl>
1 tslm    Test     173.
2 tslmvar Test     155.
```

Looking at the differences between RMSE, AIC, and Winkler score between the standard trend/season model and the trend/season model with average fuel cost added in, the additional explanatory variable does not significantly improve model output, so we can safely discard the extra explanatory variable in further analysis.

From here, we looked at TSLM models build on trend and season, polynomial trend and season, and just season as well as two ETS models, one algorithmic and one preset with additive features.

```
charge_fit <- train_ts %>%
  model(tslmseason = TSLM(average_charge ~ season()),
    tslm = TSLM(average_charge ~ trend() + season()),
        tslm2 = TSLM(average_charge ~ I(trend()^2)+ trend() + season()),
        ETSset = ETS(average_charge ~ error('A')+ trend('A')+ season('A')),
        ETSAlg = ETS(average_charge))
charge_fit %>%
  glance()%>%
  select(.model, AIC, AICc, BIC)
```

```
# A tibble: 5 × 4
  .model       AIC  AICc   BIC
  <chr>      <dbl> <dbl> <dbl>
1 tslmseason  136.  139.  143.
```

```
2 tslm        138.  142.  146.
3 tslm2       127.  132.  137.
4 ETSset      240.  249.  252.
5 ETSAlg      236.  241.  246.
```

```
charge_fit %>% forecast(test_ts)%>% accuracy(data_ts)
```

```
# A tibble: 5 × 10
  .model     .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE   ACF1
  <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
1 ETSAlg     Test  10.4   11.4  10.4  7.60  7.60  1.22 0.956 0.148
2 ETSset     Test  10.8   12.0  10.8  7.90  7.90  1.28 1.01  0.0448
3 tslm       Test  22.2   22.7  22.2 16.6  16.6   2.62 1.91  0.0445
4 tslm2      Test  -3.92  12.3  10.6 -3.28  8.03  1.25 1.04  0.710
5 tslmseason Test  20.3   20.9  20.3 15.1  15.1   2.40 1.76  0.0776
```

```
charge_fit %>% forecast(test_ts) %>%
  accuracy(data_ts, list(winkler = winkler_score))
```

```
# A tibble: 5 × 3
  .model     .type winkler
  <chr>      <chr>   <dbl>
1 ETSAlg     Test     50.6
2 ETSset     Test     52.7
3 tslm       Test    173.
4 tslm2      Test     44.1
5 tslmseason Test    167.
```

Looking at AIC, AICc, and BIC, the ETS models perform significantly worse than the others, so we decided to discard them. Amongst the remaining TSLM models, the polynomial trend model had the best RMSE and Winkler score while falling in line with the others on AIC. It looked to us like the best model would be the polynomial trend plus season. However, we also knew that data from 2020 is heavily impacted by COVID-19 which could create the illusion of a trend that was really created by extenuating circumstance. To test this, we re-split our train and test sets to exclude the year 2020 and reran the models.

```
train_ts_covidless <- filter_index(.data = data_ts, .~'2019 Q4')
test_ts_covidless <- filter_index(.data = data_ts, '2021 Q1' ~.)
```

```
charge_fit <- train_ts_covidless %>%
  model(tslmseason = TSLM(average_charge ~ season()),
    tslm = TSLM(average_charge ~ trend() + season()),
        tslm2 = TSLM(average_charge ~ I(trend()^2)+ trend() + season()),
      )


charge_fit %>%
  glance()%>%
  select(.model, AIC, AICc, BIC)
```

```
# A tibble: 3 × 4
  .model       AIC  AICc   BIC
  <chr>      <dbl> <dbl> <dbl>
1 tslmseason  84.9  89.2  89.9
2 tslm        84.4  90.9  90.4
3 tslm2       75.7  85.0  82.7
```

```
charge_fit %>% forecast(test_ts_covidless) %>%
  accuracy(data_ts, list(winkler = winkler_score))
```

```
# A tibble: 3 × 3
  .model     .type winkler
  <chr>      <chr>   <dbl>
1 tslm       Test     280.
2 tslm2      Test     207.
3 tslmseason Test     156.
```

```
charge_fit %>% forecast(test_ts_covidless)%>% accuracy(data_ts)
```

```
# A tibble: 3 × 10
  .model     .type    ME  RMSE   MAE   MPE  MAPE  MASE RMSSE  ACF1
  <chr>      <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 tslm       Test   23.1  25.6  23.7  17.6  18.1  3.22  2.42 0.497
2 tslm2      Test  -43.0  49.0  43.0 -33.6  33.6  5.85  4.63 0.790
3 tslmseason Test   14.2  17.3  15.5  10.6  11.8  2.11  1.63 0.405
```

Looking at the results with 2020 removed from the data, it appears that the TSLM model with just the season is the best model giving us the following model:

```
charge_select<- train_ts %>%
  model(tslmseason = TSLM(average_charge ~ season()))

coef(charge_select)
```

```
# A tibble: 4 × 6
  .model     term          estimate std.error statistic  p.value
  <chr>      <chr>            <dbl>     <dbl>     <dbl>    <dbl>
1 tslmseason (Intercept)     119.        3.11    38.3   2.18e-24
2 tslmseason season()year2    -6.66      4.40    -1.51  1.42e- 1
3 tslmseason season()year3   -15.3       4.55    -3.36  2.40e- 3
4 tslmseason season()year4    -4.31      4.55    -0.948 3.52e- 1
```

The P-value for quarter 3 indicates that average revenue per passenger is significantly different from quarter 1, the P-value for quarter 2 is weaker, and the P-value for quarter 4 indicates that it is not significantly different. The seasonal model is still worth using because it captures the swing between quarters 1 and 3, but we should keep in mind that not every quarter is significantly different.

Plane Fill Model

To test the fill fit, we once again built most possible combinations of models, including models that include average revenue per passenger as an additional explanatory variable. We did this because it seemed possible that if average revenue per passenger increased, less people may be willing to fly.

```
fill_fit <- train_ts %>%
  model(TSLMVar = TSLM(fill ~ trend() + season() + average_charge),
        TSLM = TSLM(fill ~ trend() + season()),
        TSLMSeason = TSLM(fill ~ season()),
        TSLMSeasonVar = TSLM(fill ~ season() + average_charge),
        ETSset = ETS(fill ~ trend('A') + error('A') + season('A')),
        ETSalg = ETS(fill))
fill_fit %>%
  glance()%>%
  select(.model, AIC, AICc, BIC)
```

```
# A tibble: 6 × 4
  .model           AIC   AICc    BIC
  <chr>          <dbl>  <dbl>  <dbl>
1 TSLMVar       -147.  -142.  -137.
2 TSLM          -132.  -129.  -124.
3 TSLMSeason    -124.  -122.  -117.
4 TSLMSeasonVar -135.  -132.  -127.
5 ETSset         -27.7  -18.7  -15.1
6 ETSalg         -38.9  -38.0  -34.7
```

```
fill_fit %>% forecast(test_ts) %>% accuracy(data_ts)
```

```
# A tibble: 6 × 10
  .model        .type       ME   RMSE    MAE     MPE  MAPE  MASE RMSSE    ACF1
  <chr>         <chr>    <dbl>  <dbl>  <dbl>   <dbl> <dbl> <dbl> <dbl>   <dbl>
1 ETSalg        Test   0.00248 0.0263 0.0211   0.199  2.59 0.255 0.192  0.350
2 ETSset        Test  -0.0243  0.0481 0.0374  -3.11   4.66 0.451 0.351  0.319
3 TSLM          Test   0.187   0.188  0.187   22.7   22.7  2.25  1.37  -0.611
4 TSLMSeason    Test   0.0500  0.0570 0.0518   6.00   6.24 0.625 0.416  0.0891
5 TSLMSeasonVar Test  -0.109   0.120  0.109  -13.5   13.5  1.32  0.876 -0.303
6 TSLMVar       Test   0.0260  0.0556 0.0459   3.11   5.59 0.554 0.406 -0.254
```

```
fill_fit %>% forecast(test_ts) %>% accuracy(data_ts, list(winkler = winkler_score))
```

```
# A tibble: 6 × 3
  .model        .type winkler
  <chr>         <chr>   <dbl>
1 ETSalg        Test    0.585
2 ETSset        Test    0.627
3 TSLM          Test    0.447
4 TSLMSeason    Test    0.480
```

```
5 TSLMSeasonVar Test    0.428
6 TSLMVar       Test    0.378
```

Winkler scores are mostly the same across models, with the model including average revenue being the best. AIC metrics are similar across TSLM models with the ETS models being significantly worse. The seasonal exclusive model as well as the average revenue model had good RMSEs as well as the ETS models. One thing to take into consideration is the fact that the model including average charge is in some sense just favoring seasonality to an extra degree. We already know that average charge is essentially just seasonal, so including it as an additional explanatory variable does not make as much sense especially considering the complexity it adds to the model. Once again, 2020 may be impacting these results, so we decided it was best to run another test with 2020 removed again.

```
fill_fit <- train_ts_covidless %>%
model(TSLMVar = TSLM(fill ~ trend() + season() + average_charge),
        TSLM = TSLM(fill ~ trend() + season()),
        TSLMSeason = TSLM(fill ~ season()),
        TSLMSeasonVar = TSLM(fill ~ season() + average_charge),
        ETSset = ETS(fill ~ trend('A') + error('A') + season('A')),
        ETSalg = ETS(fill))


fill_fit %>%
  glance()%>%
  select(.model, AIC, AICc, BIC)
```

```
# A tibble: 6 × 4
  .model           AIC   AICc    BIC
  <chr>          <dbl>  <dbl>  <dbl>
1 TSLMVar       -160.  -151.  -153.
2 TSLM          -162.  -156.  -156.
3 TSLMSeason    -146.  -142.  -141.
4 TSLMSeasonVar -147.  -140.  -141.
5 ETSset         -94.7  -76.7  -85.7
6 ETSalg         -91.1  -81.7  -84.1
```

```
fill_fit %>% forecast(test_ts) %>% accuracy(data_ts)
```

```
# A tibble: 6 × 10
  .model        .type       ME   RMSE    MAE    MPE  MAPE  MASE RMSSE  ACF1
  <chr>         <chr>    <dbl>  <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
1 ETSalg        Test   0.0168  0.0401 0.0329  1.93   4.00 0.397 0.292 0.253
2 ETSset        Test   0.0353  0.0478 0.0381  4.21   4.57 0.459 0.349 0.142
3 TSLM          Test   0.0677  0.0690 0.0677  8.23   8.23 0.816 0.504 0.316
4 TSLMSeason    Test  -0.00148 0.0230 0.0195 -0.233  2.40 0.235 0.168 0.438
5 TSLMSeasonVar Test  -0.0214  0.0304 0.0240 -2.68   2.98 0.290 0.221 0.425
6 TSLMVar       Test   0.0607  0.0622 0.0607  7.38   7.38 0.732 0.454 0.335
```

```
fill_fit %>% forecast(test_ts) %>% accuracy(data_ts, list(winkler = winkler_score))
```

```
# A tibble: 6 × 3
  .model        .type winkler
  <chr>         <chr>   <dbl>
1 ETSalg        Test    0.278
2 ETSset        Test    0.632
3 TSLM          Test    1.09
4 TSLMSeason    Test    0.100
5 TSLMSeasonVar Test    0.160
6 TSLMVar       Test    0.494
```

Looking at these results, the purely seasonal model once again appears to be the best once the shock from COVID is removed from the data. One final cross validation test was performed between just the trend/season TSLM model and the exclusively seasonal TSLM model.

```
train_ts %>% stretch_tsibble(.init = 10, .step = 2)%>%
  model(

    TSLM = TSLM(fill ~ trend() + season()),
    TSLMSeason = TSLM(fill ~ season()),
    )%>%
  forecast(h=2)%>%
  accuracy(data_ts)%>%
  select(.model, ME, RMSE, MAPE)
```

```
# A tibble: 2 × 4
  .model          ME  RMSE  MAPE
  <chr>        <dbl> <dbl> <dbl>
1 TSLM        0.0151 0.137  15.6
2 TSLMSeason -0.0611 0.138  14.3
```

The models perform practically the same, so it is better to prioritize the simpler model. This means that once again our best model is purely seasonal with the following coefficients.

```
fill_fit <- train_ts %>%
  model(
        TSLMSeason = TSLM(fill ~ season()),
       )
coef(fill_fit)
```

```
# A tibble: 4 × 6
  .model     term          estimate std.error statistic  p.value
  <chr>      <chr>            <dbl>     <dbl>     <dbl>    <dbl>
1 TSLMSeason (Intercept)     0.752     0.0406    18.5   1.75e-16
2 TSLMSeason season()year2   0.0266    0.0575     0.463 6.47e- 1
3 TSLMSeason season()year3   0.0223    0.0595     0.374 7.11e- 1
4 TSLMSeason season()year4   0.0267    0.0595     0.448 6.58e- 1
```

Something interesting can be seen when observing the p-values for the seasonal coefficients.

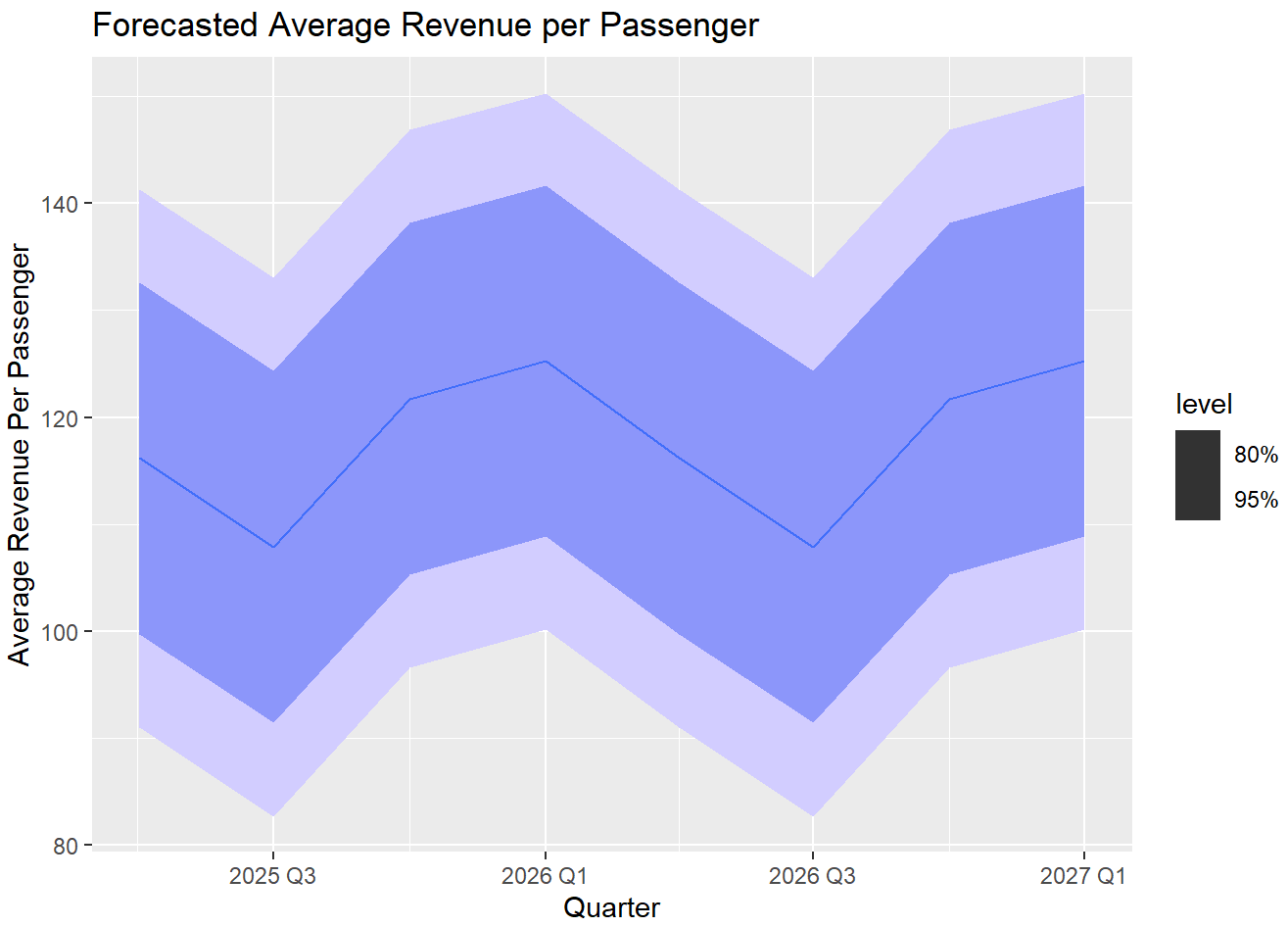
```r
fill_fit <- train_ts %>%
  model(
        TSLMSeason = TSLM(fill ~ season()),
       )
coef(fill_fit)%>%
  select(.model, term, p.value )
```

```
# A tibble: 4 × 3
  .model     term           p.value
  <chr>      <chr>            <dbl>
1 TSLMSeason (Intercept)   1.75e-16
2 TSLMSeason season()year2 6.47e- 1
3 TSLMSeason season()year3 7.11e- 1
4 TSLMSeason season()year4 6.58e- 1
```

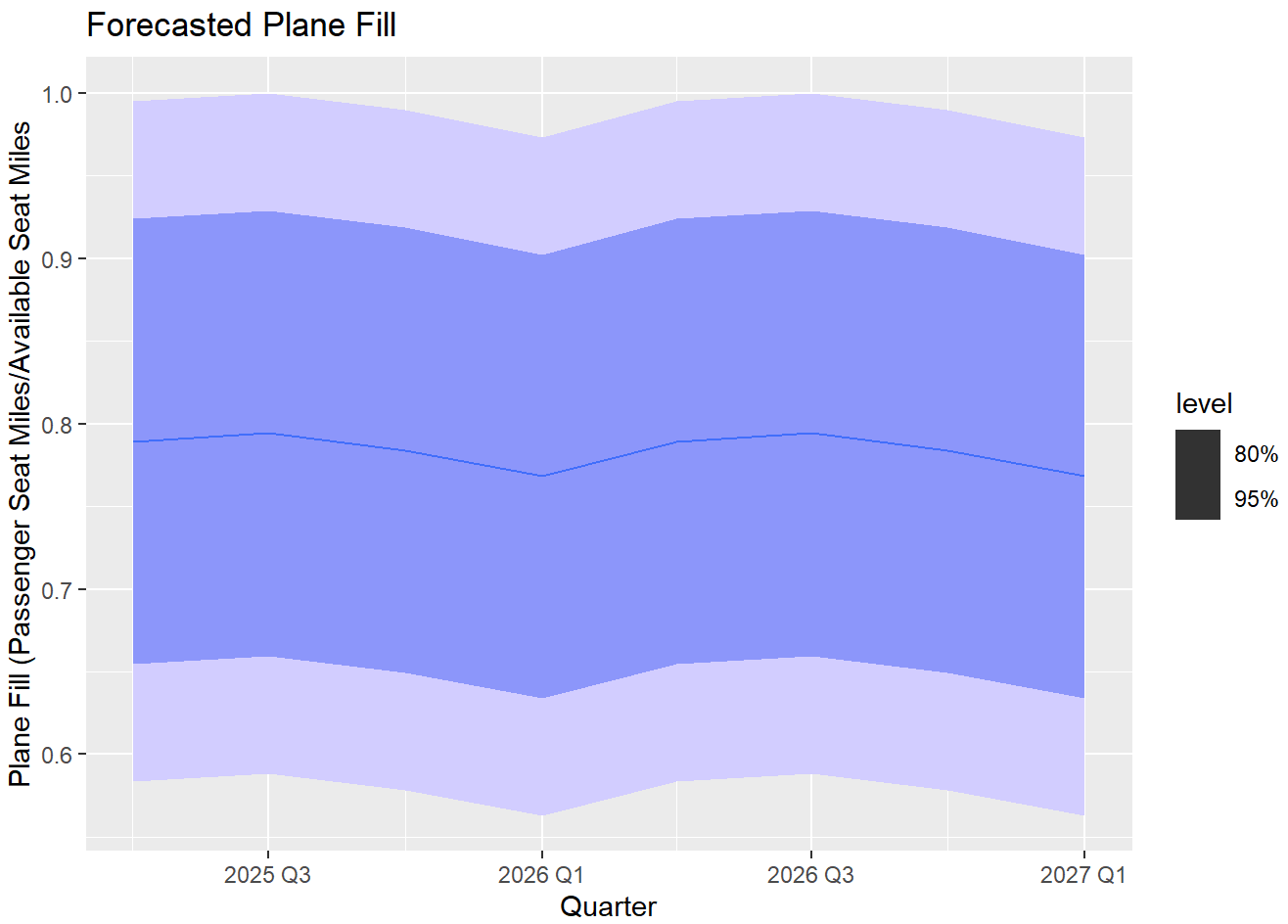
The coefficients outside of the intercept are not actually significant! This means that despite being the best predicted model, we do not have statistical evidence to show that the fill rate is significantly different from quarter to quarter. This has good and bad business implications, but first we should run our forecasts.

Forecasting

```r
charge_select <- data_ts%>%
  model(TSLMseason = TSLM(average_charge ~ season()))

charge_select%>% forecast(h=8)%>% autoplot()+
  labs(title = 'Forecasted Average Revenue per Passenger', y = 'Average Revenue Per Passenger')
```



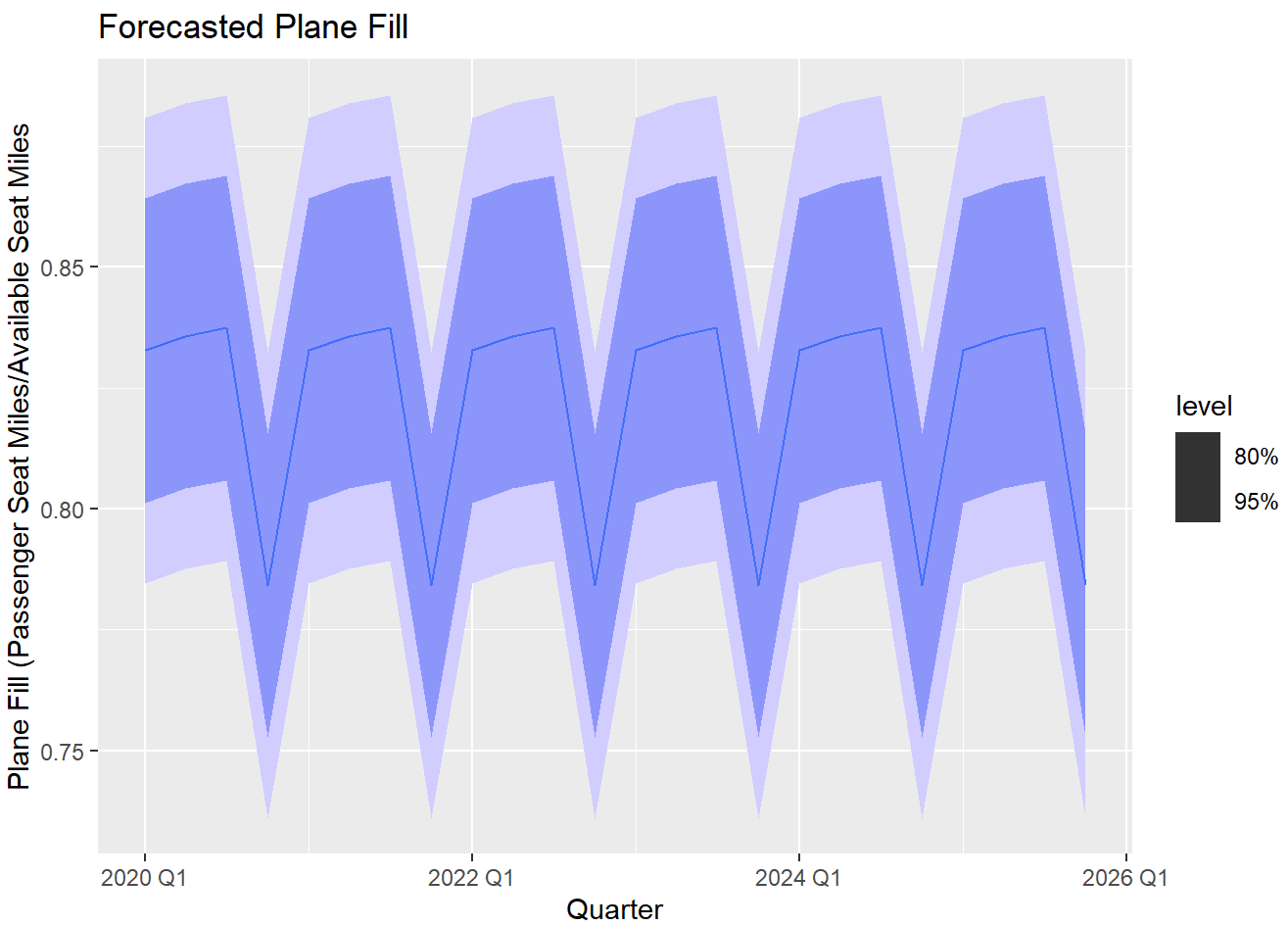

```
fill_select <- data_ts%>%
   model(
        TSLMSeason = TSLM(fill ~ season()),
       )
fill_select %>% forecast(h = 8) %>% autoplot()+
  labs(title = 'Forecasted Plane Fill', y = 'Plane Fill (Passenger Seat Miles/Available Seat Mile
```



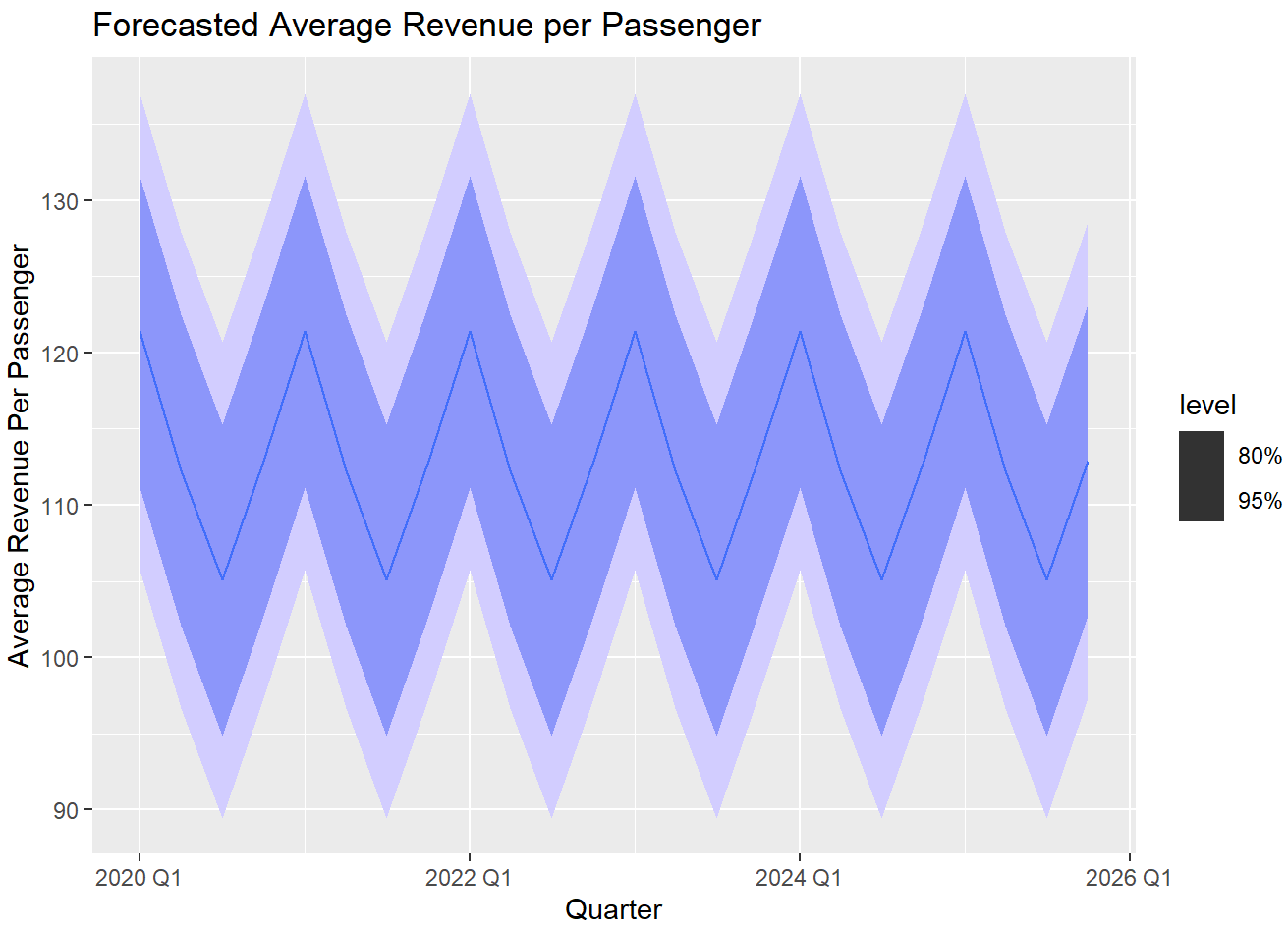
There are a few implications to the two forecasts. First is that barring extenuating circumstances like COVID, Allegiant can expect some stability within revenue per passenger and plane fill rate. For plane fill rate, this is good because it indicates that however they are currently scheduling available seat miles effectively matches actual passenger seat miles without risking overbooking. Revenue per passenger is more of a mixed bag. Without the presence of a significant trend, it means that if Allegiant wants to increase their revenue, they must increase their passenger count as there is no expected revenue increase within their current passenger base. While the prediction interval for forecasted plane fill is broad, this is largely in part due to the fact that the final model was made off of all the data in the set which included all of the COVID downturns, diminishing confidence. However, this also reveals the risk associated with these two metrics. Average revenue per passenger is only associated with seasonality and average fill rate isn't really associated with anything measurable during normal operations. However, both are significantly effected by random world events such as COVID-19. It would be extremely difficult for Allegiant to predict events such as disease outbreaks, terrorist threats, or anything else that may dissuade people from flying, but it could prepare by looking at the downturns from events like COVID and determining what resources they need to set aside to be able to cope with such an event until revenues and plane fill return back to their normal seasonal baselines.

The following forecasts are intended to show what the forecast would look like if the COVID data were removed. These have much tighter prediction intervals, indicating that the four data points disrupted by COVID were creating uncertainty in the model.

```
fill_select <- train_ts_covidless%>%
   model(
        TSLMSeason = TSLM(fill ~ season()),
       )
fill_select %>% forecast(h = 24) %>% autoplot()+
  labs(title = 'Forecasted Plane Fill', y = 'Plane Fill (Passenger Seat Miles/Available Seat Mile
```




```
charge_select <- train_ts_covidless%>%
  model(TSLMseason = TSLM(average_charge ~ season()))

charge_select%>% forecast(h=24)%>% autoplot()+
  labs(title = 'Forecasted Average Revenue per Passenger', y = 'Average Revenue Per Passenger')
```


Forecasted Average Revenue per Passenger
Average Revenue Per Passenger
130
120
110
100
90
2020 Q1
2022 Q1
2024 Q1
2026 Q1
Quarter
level
80%
95%